

Studi'OS

Roadmap - Project AI Bootstrap

Brancher un nouveau projet à Studi'OS et le rendre immédiatement exploitable par Claude Code, OpenCode, Codex ou un autre harness, sans recopier manuellement une configuration IA complète.

Vision : un projet est enregistré dans Studi'OS, le poste local reçoit un bootstrap minimal, les ressources communes sont résolues à la demande, puis le harness peut reprendre une tâche via `studio_prepare_context`.

Principes directeurs

- Studi'OS reste agnostique au modèle, au provider et au harness.
- Aucun LLM ni agent n'est hébergé sur le VPS.
- Le VPS conserve l'état partagé ; le daemon/CLI local effectue les écritures dans le dépôt local.
- Le bootstrap permanent doit rester très petit ; le contexte est chargé à la demande.
- `.agents/` reste la source canonique locale lorsque des ressources projet sont nécessaires.
- Library + Resolution fournissent les ressources communes et les bindings.
- Les projections Claude/OpenCode/Codex sont générées, jamais maintenues comme sources indépendantes.
- Aucune configuration ne doit écraser silencieusement des fichiers utilisateur existants.

1. Expérience utilisateur cible

Le parcours cible doit être suffisamment simple pour qu'un développeur n'ait pas à connaître l'organisation interne des rules, skills, adapters ou bindings.

Étape	Expérience attendue
1	Créer ou cloner un projet (ex. projet Godot).
2	Ajouter/enregistrer le projet dans Studi'OS.
3	Choisir « Configurer l'intégration IA » ou lancer l'équivalent CLI local.
4	Studi'OS résout les ressources communes, projet et runtime applicables.
5	Le daemon/CLI local génère uniquement le bootstrap et les projections nécessaires.
6	L'utilisateur ouvre Claude/OpenCode/Codex à la racine du projet.
7	L'agent utilise studio_prepare_context puis charge les ressources spécialisées à la demande.
8	Studi'OS peut ensuite détecter le drift et proposer une resynchronisation.

2. Architecture cible

Cloud / VPS : projets, Library, versions, bindings, Runtime Registry, Resolution, tâches, décisions et contexte partagé. Local : daemon/CLI, inspection du dépôt, écriture des fichiers de bootstrap et génération des projections. Harness : consomme le bootstrap minimal puis demande son contexte à Studi'OS.

Flux : Dashboard/API → configuration désirée → Library/Bindings/Resolution → daemon ou CLI local → génération sûre dans le dépôt → Claude/OpenCode/Codex → studio_prepare_context.

P0 - Architecture Gate & inventaire

- Auditer les briques existantes : projets, daemon, adapters, Library, Resolution, bindings et prepare_context.
- Définir précisément ce qui est global, spécifique au projet, généré, versionné et local-only.
- Décider le contrat de bootstrap sans coder : manifest/config minimal, ownership des fichiers, stratégie de drift.
- Vérifier qu'aucune nouvelle abstraction ne duplique Library, Resolution ou les adapters existants.
- Livrable : DEC/architecture + matrice CURRENT / REUSE / GAP / DECISION.

P1 - Contrat Project AI Bootstrap

- Introduire un contrat déclaratif minimal décrivant l'intégration IA souhaitée d'un projet.
- Le contrat référence des stable keys/versions/bindings ; il ne copie pas le contenu complet des skills/rules.
- Définir les harnesses ciblés, capacités requises, ressources projet et politique de génération.
- Prévoir un mode dry-run et une représentation explicite des conflits.
- Aucune écriture filesystem distante depuis le VPS.

P2 - Résolution du bundle projet

- Composer un « bootstrap plan » déterministe à partir du projet, de la Library, des bindings et de Resolution.
- Séparer ressources communes Studi'OS et ressources propres au projet.
- Retourner provenance complète : pourquoi chaque rule/skill/agent/runtime est sélectionné.
- Fail-closed pour références inconnues ou incompatibilités critiques.
- Tests de déterminisme et d'agnosticisme provider/model/harness.

P3 - Générateur local sûr

- Implémenter côté daemon/CLI l'application du bootstrap plan au dépôt local.
- Créer uniquement les fichiers nécessaires : CLAUDE.md minimal et projections supportées.
- Ne jamais écraser silencieusement un fichier utilisateur : check, diff, backup/confirmation selon politique.
- Utiliser les adaptateurs existants pour Claude/OpenCode/Codex ; pas de logique harness dans le Core.
- Supporter init, check, diff/dry-run et sync.

P4 - Ressources projet et héritage Library

- Permettre à un projet de déclarer ses propres rules/skills/AgentDefinitions sous .agents/ lorsque nécessaire.
- Résoudre l'héritage : Studio/global → projet → session selon les précédences déjà établies.
- Éviter de recopier studio-context, studio-task, studio-decision et studio-handoff dans chaque dépôt si la Library peut les fournir.
- Garantir qu'un nouveau projet peut rester presque vide côté configuration locale.

P5 - Drift detection & resynchronisation

- Détecter projection absente, obsolète, modifiée ou incompatible.
- Réutiliser/étendre l'anti-drift existant plutôt que créer un second système.
- Afficher un statut : configuré / incomplet / drift / incompatible.
- Fournir sync explicite avec aperçu des changements avant écriture.

P6 - Dashboard / UX

- Ajouter au projet une section « Intégration IA » simple et progressive.
- Afficher harnesses configurés, statut local, ressources actives, runtime/binding et provenance utile.
- Actions : préparer la configuration, afficher instructions locales, vérifier, resynchroniser.
- Le Dashboard ne prétend jamais avoir écrit sur un poste hors ligne ; l'exécution réelle reste locale.

P7 - Onboarding d'un projet neuf

- Scénario E2E : projet Godot vierge → ajout à Studi'OS → bootstrap local → lancement Claude → reprise d'une tâche.
- Le nouvel agent doit obtenir le contexte pertinent avec studio_prepare_context sans charger une documentation massive.
- Tester ensuite OpenCode/Codex depuis le même projet sans dupliquer la source canonique.
- Mesurer fichiers générés, tokens permanents et nombre d'actions manuelles.

P8 - Multi-machine & équipe

- Tester un second développeur qui clone le même dépôt sur une autre machine.
- La configuration partagée doit être reproductible sans embarquer chemins absolus, secrets ou état machine.
- Les éléments machine-specific restent locaux et sont reconstruits depuis Studi'OS.
- Valider offline/reconnexion et absence de dépendance LAN.

P9 - Durcissement & documentation

- Sécurité Git/filesystem, idempotence, conflits, rollback et compatibilité versions.
- Documentation utilisateur courte : « connecter un projet », « ajouter un harness », « réparer le drift ».
- Documentation technique centrée sur contrats et ownership, pas sur des procédures copiées.
- Gate final : aucune régression Agent Integration et aucune dépendance à Claude.

3. Fichiers et responsabilités - cible

Élément	Rôle	Ownership
CLAUDE.md	Bootstrap Claude minimal ; aucune documentation lourde.	Généré/projet, selon politique
.agents/	Ressources réellement spécifiques au projet.	Source canonique locale projet
.claude/	Projection Claude.	Générée par adapter
.opencode/	Projection OpenCode.	Générée par adapter
.codex/	Projection Codex.	Générée par adapter
Library	Ressources communes/versionnées et réutilisables.	Studi'OS
Bindings/Runtime	Sélection runtime/harness/provider/model sans pins dans les dépôts	Studi'OS
Daemon/CLI	Écriture locale, check, sync, drift.	Client local Studi'OS

4. Gates de sécurité

- Pas d'écriture locale depuis le VPS ; uniquement daemon/CLI autorisé sur la machine.
- Pas de secret, token, chemin utilisateur absolu ou état machine dans les fichiers partagés.
- Pas d'écrasement silencieux d'un CLAUDE.md/.agents existant.
- Dry-run/diff disponible avant synchronisation.
- Source canonique unique ; projections régénérables et contrôlées anti-drift.
- Aucune dépendance obligatoire à Claude : OpenCode/Codex doivent utiliser le même cœur.
- Le bootstrap permanent reste minimal et mesuré.
- studio_prepare_context reste la porte d'entrée normale vers le contexte projet.

5. Critères d'acceptation finaux

- ✓ Un projet neuf peut être enregistré puis préparé sans copier manuellement une configuration IA existante.
- ✓ Claude Code peut démarrer à la racine et reprendre une tâche avec un bootstrap minimal.
- ✓ OpenCode et Codex peuvent être ajoutés sans recréer rules/skills/agents.
- ✓ Un second poste peut reconstruire sa configuration locale depuis les mêmes sources partagées.
- ✓ Le drift est détecté et réparable de manière explicite.
- ✓ Les ressources communes Studi'OS ne sont pas dupliquées inutilement dans chaque dépôt.
- ✓ Aucun LLM n'est exécuté sur le VPS.
- ✓ Les tests prouvent l'agnosticisme et la continuité Agent A → Agent B.

6. Hors périmètre

- Créer un IDE ou remplacer Git.
- Héberger des modèles/agents sur le VPS.
- Synchroniser tout le filesystem d'un projet.
- Introduire un second moteur de résolution ou un second système de Library.
- Copier automatiquement toutes les rules/skills dans tous les projets.

- Masquer les conflits en écrasant les fichiers locaux.

Définition de réussite : « J'ajoute un projet à Studi'OS, je configure ce poste, j'ouvre mon harness et je peux travailler immédiatement ; Studi'OS fournit le contexte et les ressources nécessaires sans que je maintienne manuellement une configuration IA par projet. »